

Interactive Seminar Activity: Manual vs. AI-Assisted API Testing

Part 1: Environment Setup & Quick Smoke Test (2 minutes)

1. Verification Checklist:

- Ensure the demo backend is running locally:

```
cd demo-src
npm install
npm start
```

(The API server should be listening at: <http://localhost:3000>)

- Open the **Postman** application.
- Ensure you have imported [Activity_Collection.json](#) and [Activity_Environment.json](#) from the [demo-src/postman/](#) directory.
- Set your active Postman environment to "**Activity Env**" (top-right selector).

2. Retrieve Authentication Token:

- Send a **POST** [/api/auth/login](#) request.
 - *Payload:* Uses pre-configured environment variables `{{username}}` and `{{password}}` (which resolve to `alice` and `password123`).
- Copy the `token` value returned in the JSON response.
- Click the **Environment quick look** (eye icon in the top right) or edit the "**Activity Env**" environment, and paste the copied token into the **Current Value** of the `auth_token` variable.
 - *Note:* If Postman displays a "**Secrets Detected**" warning, click the **Action** button next to the field, select **Override** (or **Ignore**), and press **Ctrl + S** to save the changes.
- (This token is used by downstream requests via the Authorization header: `Bearer {{auth_token}}`)

3. Verify API Availability:

- Send a **GET** [/api/products](#) request.
- If you receive a **200 OK** status code with a JSON list of 7 products → **Your environment is ready!**
- If you face connection issues, consult the facilitator immediately.

Part 2: Endpoint Under Test & Specifications

You will design and run tests for the Coupon Redemption API endpoint: **POST** [/api/coupon/redeem](#)

1. OpenAPI Specification Snippet:

```
paths:
  /api/coupon/redeem:
    post:
      summary: Redeem a discount coupon
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required:
                - coupon_code
                - user_id
              properties:
                coupon_code:
                  type: string
                  example: "SAVE20"
                user_id:
                  type: integer
                  example: 1
      responses:
        '200':
          description: Coupon applied successfully
          content:
            application/json:
              schema:
                type: object
                properties:
                  success:
                    type: boolean
                  discount_percent:
                    type: number
                  message:
                    type: string
        '400':
          description: Invalid or expired coupon code
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
        '409':
          description: Coupon already redeemed by this user
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
```

2. Core Business Rules (Not explicitly modeled in the OpenAPI schema):

1. **Single Redemption per User:** A user (`user_id`) can only redeem a specific `coupon_code` once. Subsequent redemptions must return **409 Conflict**.
2. **Expiration Check:** The API checks the current date against the coupon's `expiry_date` stored in the system. Expired coupons must return **400 Bad Request**.
3. **Active Coupons in Database:**

◦ `SAVE20` (Discount: 20%, Expiry Date: 2026-12-31, Active)

◦ `HALF` (Discount: 50%, Expiry Date: 2026-12-31, Active)

◦ `EXPIRED10` (Discount: 10%, Expiry Date: 2024-01-01, **Expired**)

Part 3: Test Generation Challenge (10 minutes)

GROUP A: MANUAL SCRIPT WRITING

- **Instruction:** Open the request `POST /api/coupon/redeem` → Go to the **Scripts** tab → Go to **Post-response** (or **Tests**). Write tests in plain JavaScript using `pm.test()`.
- **Task:** Cover at least **5 test scenarios** (happy paths, validation errors, and business rule violations). Document them below:

#	Test Scenario / Input	Expected Status	Key Assertion Check (JavaScript / Logic)
1			
2			
3			
4			
5			

Example code for testing: **Common Postman Commands:**

Command / Assertion	Description
<code>pm.environment.set("key", value)</code>	Creates or updates a variable in your active Environment. Useful for variable chaining.
<code>pm.environment.get("key")</code>	Retrieves the value of an existing Environment variable.
<code>pm.response.json()</code>	Parses the raw response body into a readable JavaScript object/array.
<code>pm.test("Name", function() {...})</code>	Wraps your validation logic. The "Name" will appear in your Postman test result reports.

Command / Assertion	Description
<code>pm.response.to.have.status(200)</code>	A quick assertion to verify the HTTP status code returned by the server.
<code>pm.expect(A).to.eql(B)</code>	Asserts that value A is strictly equal to value B.
<code>pm.expect(obj).to.have.property('id')</code>	Validates that a specific key (e.g., 'id') exists within a JSON object.
<code>pm.expect(obj.id).to.be.a('number')</code>	Checks the exact data type of a value (can be 'number', 'string', 'boolean').
<code>pm.expect(arr).to.be.an('array').that.is.not.empty</code>	Ensures the returned data is an array and contains at least one item.
<code>pm.expect(time).to.be.below(800)</code>	Asserts that a numeric value is less than a specified target (great for performance testing).

GROUP B: AI-ASSISTED GENERATION

- Instruction:**

1. Open the request `POST /api/coupon/redeem` and send a request with a valid body (e.g., `{"coupon_code": "SAVE20", "user_id": 1}`).
2. Go to the **Scripts / Post-response** tab → Click **"Ask Postbot"** or click Generate Tests on Test Result tab on the bottom right.
3. Type the prompt:

"Generate tests for this coupon redeem endpoint. Cover status codes, response body validation, and edge cases."

4. (If Postbot is out of credits: Copy the OpenAPI spec from Part 2 and paste it into ChatGPT/Claude with the same prompt, then paste the generated JavaScript into the Postman test tab).

- Task:** Review the generated tests and complete the table below:

#	AI-Generated Test Name	Main Assertion Generated	Did the AI test business logic like coupon expiry or double-redemption?	Did you need to manually edit/fix the code? (Why?)
1				
2				
3				
4				
5				

Part 4: Peer Discussion & Comparison (5 minutes)

Find a partner from the opposite group (Group A joins Group B) and complete the comparison grid below:

Comparison Metric	Group A (Manual)	Group B (AI / Postbot)
Total Test Cases Generated	_____ cases	_____ cases
Time taken to complete (Minutes)	_____ mins	_____ mins
Assertion Code Accuracy	☐ Syntax-error free ☐ Required code fixes	☐ Syntax-error free ☐ Required code fixes
Checks HTTP 200 (Success path)	☐ Yes ☐ No	☐ Yes ☐ No
Checks HTTP 409 (Double-redemption)	☐ Yes ☐ No	☐ Yes ☐ No
Checks HTTP 400 (Expired coupon)	☐ Yes ☐ No	☐ Yes ☐ No
Hallucinated Attributes (AI asserting non-existent fields like user_name, expiry_time)	☐ Yes ☐ No	☐ Yes ☐ No

Part 5: Gap Identification & Reflection (3 minutes)

Discuss with your partner and answer the following questions:

1. **Business Logic Gap:** Did the AI identify and test the scenario for the expired coupon (**EXPIRED10**)? Why or why not?

◦ Response:

◦
2. **Hallucination Gap:** Did the AI assert any fields that do not exist in either the OpenAPI specification or the actual API response? (e.g., trying to read **responseJson.expiry_date** or **responseJson.user.username**)?

◦ Response:

◦
3. **Key Takeaway:** What is the most effective approach for combining manual human validation and AI test assistants in real-world software testing?

◦ Response:

◦

ANSWER KEY (FOR SEMINAR FACILITATORS)

1. Standard 5 Test Cases for **/api/coupon/redeem** (Postman JS):

- **Test Case 1: Redeem Valid Coupon (200 OK)**

```
pm.test("Redeem valid coupon successfully - Status 200", function () {  
  pm.response.to.have.status(200);  
  const jsonData = pm.response.json();  
  pm.expect(jsonData.success).to.be.true;  
  pm.expect(jsonData.discount_percent).to.be.a('number');  
  pm.expect(jsonData.message).to.include("applied successfully");  
});
```

- **Test Case 2: Non-existent Coupon Code (400 Bad Request)** (Send payload with "coupon_code": "INVALID_CODE")

```
pm.test("Non-existent coupon - Status 400", function () {  
  pm.response.to.have.status(400);  
  const jsonData = pm.response.json();  
  pm.expect(jsonData.error).to.equal("Coupon not found");  
});
```

- **Test Case 3: Expired Coupon Code (400 Bad Request)** (Send payload with "coupon_code": "EXPIRED10")

```
pm.test("Expired coupon - Status 400", function () {  
  pm.response.to.have.status(400);  
  const jsonData = pm.response.json();  
  pm.expect(jsonData.error).to.equal("Coupon has expired");  
});
```

- **Test Case 4: Double Redemption by Same User (409 Conflict)** (Send successfully once, then immediately trigger a second time)

```
pm.test("Double redemption - Status 409", function () {  
  pm.response.to.have.status(409);  
  const jsonData = pm.response.json();  
  pm.expect(jsonData.error).to.equal("Coupon already redeemed by this user");  
});
```

- **Test Case 5: Missing Required Request Payload Fields (400 Bad Request)** (Send payload without coupon_code or user_id)

```
pm.test("Missing fields validation - Status 400", function () {  
  pm.response.to.have.status(400);
```

```
const jsonData = pm.response.json();  
pm.expect(jsonData.error).to.include("are required");  
});
```

2. Expected AI/Postbot Gaps in this Session:

1. **Undocumented Logic Check:** Since the OpenAPI spec does not explain which codes are expired or specify the expiry date format within the schema definition, the AI cannot guess that "EXPIRED10" is expired. It only generates happy path assertions based on structural schemas.
2. **Stateful Behavior Limitation:** AI tests typically only analyze a single transaction at a time. It cannot deduce that sending two requests consecutively will mutate the state on the server and trigger a **409 Conflict** on the second attempt, unless explicitly prompted.
3. **Hallucinations:** AI assistants frequently assert structural fields like `pm.expect(jsonData.coupon_code).to.equal(...)` or `pm.expect(jsonData.user.id).to.exist` that are never returned by the API response (which only returns `success`, `discount_percent`, and `message`).